Chapter 1: Introduction of software quality assurance

Reference

This chapter refers from the book:

Mastering Software Quality Assurance: Best Practices, Tools and Techniques for Software Developers

Introduction to Software Testing

Outline

1.1. What is software?

1.2. Classification of the causes of software errors

1.3. Software quality assurance – definition and objective

1.4. Software quality factors.

1.5. Factors affecting intensity of quality assurance

Outline

1.1. What is software?

1.2. Classification of the causes of software errors

1.3. Software quality assurance – definition and objective

1.4. Software quality factors.

1.5. Factors affecting intensity of quality assurance

Outline

1.1. What is software?

1.2. Classification of the causes of software errors

1.3. Software quality assurance – definition and objective

1.4. Software quality factors.

1.5. Factors affecting intensity of quality assurance

What is software

Definition of software is really not simple. Simply code?

According to the IEEE:

- Software is: Computer programs, procedures, and possibly associated documentation and data pertaining to the operation of a computer system.
- **A 'similar definition comes from ISO:**

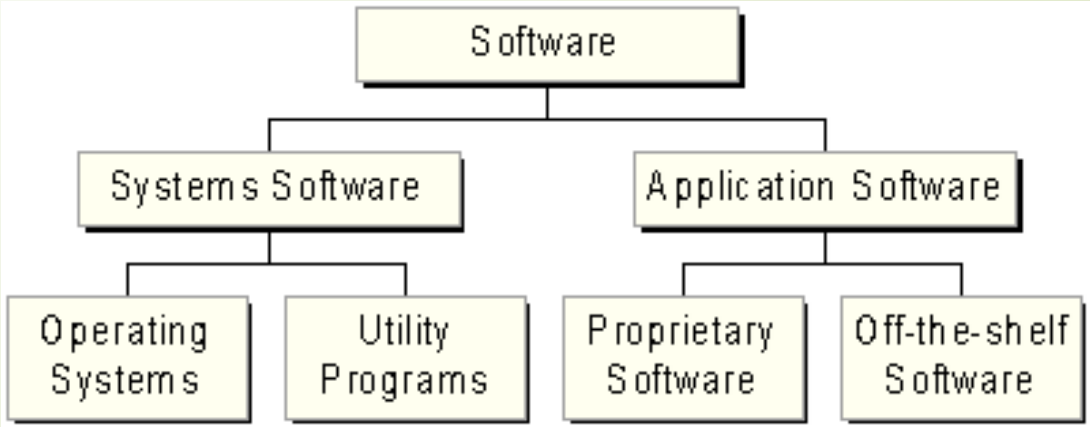
ISO definition (from ISO 9000-3) lists four components necessary to assure the quality of the software development process and years of maintenance:

- Computer programs (code)
- Procedures
- Documentation
- Data necessary for operating the software system.

Overview of Software

There are two basic types of software:

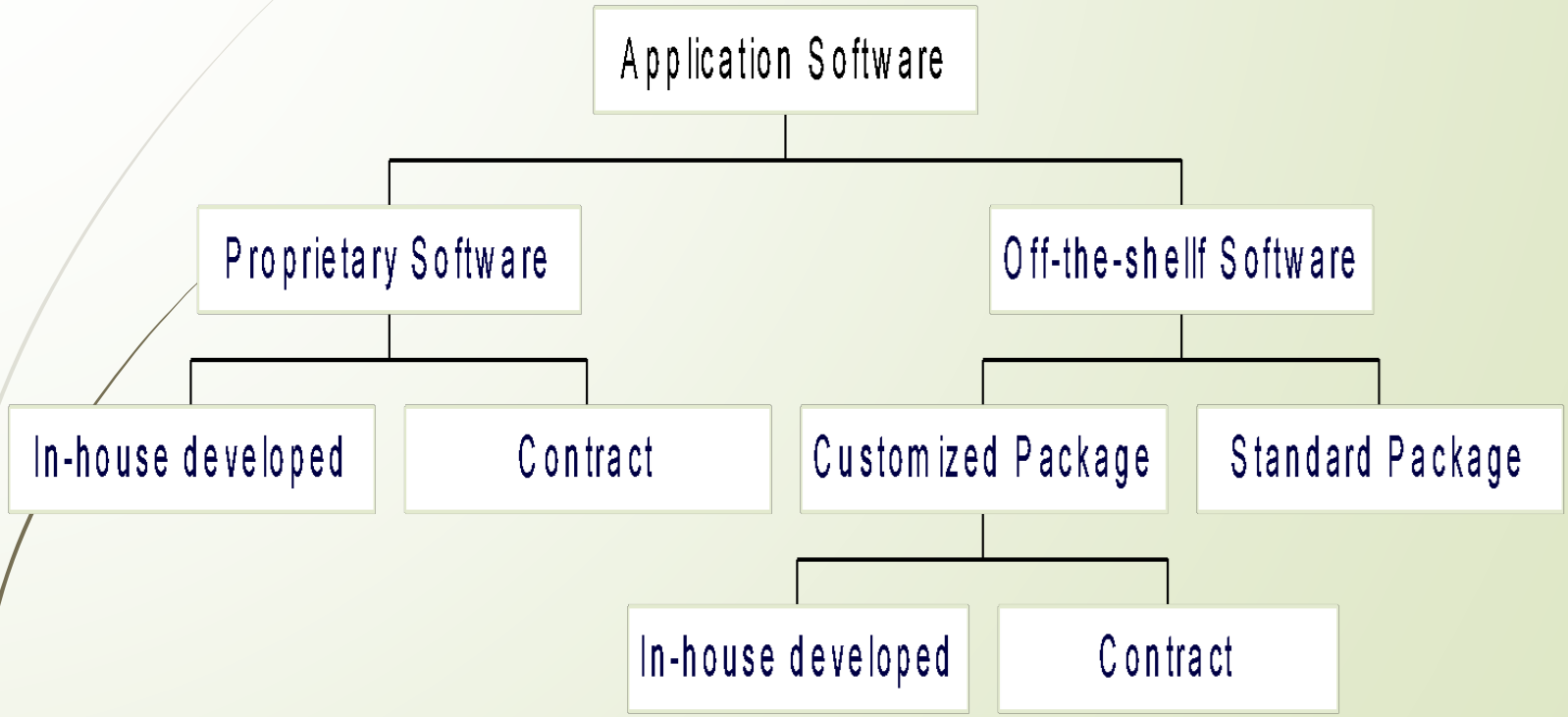
- ***Systems Software*** is a set of programs that support the computer system by coordinating the activities of the hardware and the applications. Systems software is written for a specific set of hardware, most particularly the CPU.
- ***Application Software*** is a set of programs that solve specific user-oriented problems.



Operating Systems

- An Operating System is a set of computer programs that control the computer hardware and act as an interface with application programs.
- Operating System Activities:
  - Perform common computer hardware functions like storing data on disk
  - Provide the user interface like the Windows XP Graphical User Interface
  - Provide hardware independence by serving as the interface between the application program and the hardware
  - Manage system memory to control how memory is accessed and used
  - Manage processing tasks like enabling the user to run more than one application (multitasking)
  - Control access to system resources by providing functions like password protection

Types of Application Software



Outline

1.1. What is software?

1.2. Classification of the causes of software errors

1.3. Software quality assurance – definition and objective

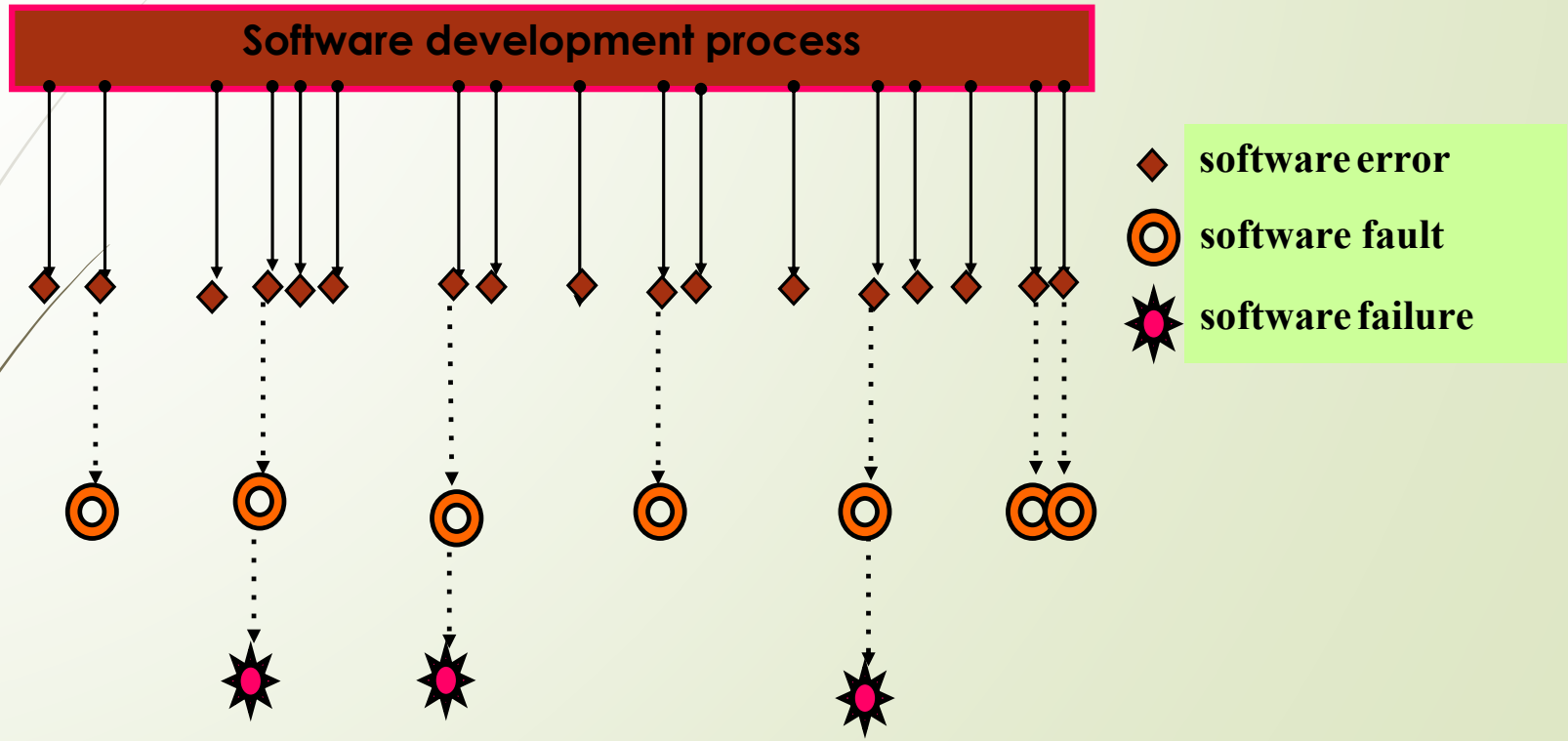
1.4. Software quality factors.

1.5. Factors affecting intensity of quality assurance

Classification of the causes of software errors

- **Software Error – made by programmer**
  - Syntax (grammatical) error
  - Logic error (multiply vice add two operands)
- **Software Fault –**
  - All software errors may **not** cause software faults
  - That part of the software may not be executed
  - (An error is present but not encountered….)
- **Software Failures – Here's the interest.**
  - A software fault becomes a software failure when/if it is **activated**.
  - Faults may be found in the software due to the way the software is executed or other constraints on the software's execution, such as execution options.

Classification of the causes of software errors



Causes of Software Error

1. Faulty requirements definition
   - Usually considered the root cause of software errors
   - Incorrect requirement definitions
     - Simply stated, 'wrong' definitions (formulas, etc.)
   - Incomplete definitions
     - Unclear or implied requirements
   - Missing requirements
     - Just flat-out 'missing.' (e.g. Program Element Code)
   - Inclusion of unneeded requirements
     - (many projects have gone amuck for including far too many requirements that will never be used.
     - Impacts budgets, complexity, development time, …

Causes of Software Error

2. Client-developer communication failures
   - Misunderstanding of instructions in requirements documentation
   - Misunderstanding of written changes during development.
   - Misunderstanding of oral changes during development.
   - Lack of attention
     - to client messages by developers dealing with requirement changes and
     - to client responses by clients to developer questions
   - Very often, these very talented individuals come from different planets, it seems.
   - Clients represent the users; developers represent a different mindset entirely some times!

Causes of Software Error

3. Deliberate deviations from software requirements
   - Developer reuses previous / similar work to save time.
   - Often reused code needs modification which it may not get or contains unneeded / unusable extraneous code.
- Book suggests developer(s) may overtly omit functionality due to time / budget pressures.
  - Another BAD choice; System testing will uncover these problems to everyone's dismay!
  - I have never seen this done intentionally!
- Developer inserting unapproved 'enhancements' (perfective coding; a slick new sort / search….); may also ignore some seemingly minor features, which sometimes are quite major.
  - Have seen this and it too causes problems and embarrassment during reviews.

Causes of Software Error

- 4. Logical design errors
  - Definitions that represent software requirements by means of erroneous algorithms.
    - Yep! Wrong formulas; Wrong Decision Logic Tables; incorrect text; wrong operators / operands…
  - Process definitions: procedures specified by systems analyst not accurate reflection of the business process specified.
    - Note: all errors are not necessarily software errors.
    - This seems like a <u>procedural error</u>, and likely not a part of the software system…
  - Erroneous Definition of Boundary Condition – a common source of errors
    - The "absolutes" like 'no more than' "fewer than," "n times or more;" "the first time," etc.

Causes of Software Error

- 4. Logical design errors (continued)
  - Omission of required software system states

    If rank is >= O1 and RPI is numeric, then….easy to miss action based on the software system state.

  - Omission of definitions concerning reactions to illegal operation of the software system.

    Including code to detect an illegal operation but failure to design the computer software reaction to this: Gracefully terminate, sound alarm, etc.

Causes of Software Error

5. Coding errors
   - Too many to try to list.
     - Syntax errors (grammatical errors)
     - Logic errors (program runs; results wrong)
     - Run-time errors (crash during execution)

Causes of Software Error

6. Non-compliance w/documentation & coding instructions
   - Non-compliance with published templates (structure)
   - Non-compliance with coding standards (attribute names…)
   - (Standards and Integration Branch)
     - Size of program;
       - Other programs must be able to run in environment!
       - Data Elements and Codes: AFM 300-4;
       - Required documentation manuals and operating instructions; AFDSDCM 300-8, etc…
   - SQA Team: testing not only execution software but coding standards; manuals, messages displayed; resources needed; resources named (file names, program names,…)

Causes of Software Error

7. Shortcomings of the Testing Process
   - Likely the part of the development process cut short most frequently!
   - Incomplete test plans
     - Parts of application not tested or tested thoroughly!
   - Failure to document, report detected errors and faults
     - So many levels of testing….we will cover.
   - Failure to quickly correct detected faults due to unclear indications that there 'was' a fault
   - Failure to fix the errors due to time constraints
     - Many philosophies here depending on severity of the error.

Causes of Software Error

8. User interface and procedure errors

8. Documentation errors
   - Errors in the design documents
   - Trouble for subsequent redesign and reuse
   - Errors in the documentation within the software for the User Manuals
   - Errors in on-line help, if available.
   - Listing of non-existing software functions
   - Planned early but dropped; remain in documentation!
   - Many error messages are totally meaningless

Causes of Software Error

The nine causes of software errors are:

1. Faulty requirements definition
2. Client-developer communication failures
3. Deliberate deviations from software requirements
4. Logical design errors
5. Coding errors
6. Non-compliance with documentation and coding instructions
7. Shortcomings of the testing process
8. User interface and procedure errors
9. Documentation errors

- You should be conversant with these

Outline

1.1. What is software?

1.2. Classification of the causes of software errors

1.3. Software quality assurance – definition and objective

1.4. Software quality factors.

1.5. Factors affecting intensity of quality assurance

Software Quality

- Software quality is:

(1) The degree to which a system, component, or process meets specified requirements.

- **by Philip Crosby**

(2) The degree to which a system, component, or process meets customer or user needs or expectations.

- **by Joseph M. Juran**

Now, more closely…

Software Quality

Software quality is:

(1) The degree to which a system, component, or process meets specified requirements.

- Seems to emphasize the specification, assuming the customer has articulated all that is needed in the specs AND that if the specs are met, the customer will be satisfied.
- I have found that this is not necessarily the case, that, if fact, often 'austere' systems are first deployed (errors discovered in specs sometimes very serious);
- customers acquiesce to the deployment with understanding of a follow-on deployment.

Software Quality

Software quality is: (Joseph Juran)

(2) The degree to which a system, component, or process meets customer or user needs or expectations.

Here, emphasis is on a satisfied customer whatever it takes. Implies specs may need corrections

But this seems to free the customer from 'professional responsibility' for the accuracy and completeness of the specs!

Assumption is that real needs can be articulated during development. This may occur, but in fact major problems can be discovered quite late. Not a happy customer!

Software Quality

Software Quality Assurance is:

1. A planned and systematic pattern of all actions necessary to provide adequate confidence that an item or product conforms to established technical requirements.

1. A set of activities designed to evaluate the process by which the products are developed or manufactured. Contrast with: quality control.

More closely:

SQA - IEEE definition

Says to plan and implement systematically!

Shows progress and instills confidence software is coming along

Refers to a software development process a methodology; a way of doing things;

Refers to the specification of technical requirements must have these.

Note that SQA must include not only process for development but for

(hopefully) years of maintenance. So, we need to consider quality issues

affecting not only development but also maintenance into overall SQA concept.

SQA activities must also include scheduling and budgeting.

SQA must address issues that arise when time constraints are encountered

– are features eliminated? Budget constraints may force compromise

SQA - Expanded Definition

Software quality assurance is:

A systematic, planned set of actions necessary to provide adequate confidence that the software development process or the maintenance process of a software system product conforms to established functional technical requirements as well as with the managerial requirements of keeping the schedule and operating within the budgetary confines.

Software Quality Assurance vs. Software Quality Control different objectives

- **Quality Control** is defined as a designed to evaluate the quality of a set of **activities** developed or manufactured product
  - We have QC inspections during development and before deployment
  - QC activities are only a **part** of the total range of QA activities.
- **Quality Assurance's** objective is to minimize the **cost of guaranteeing quality** by a **variety of activities performed throughout the development / manufacturing processes / stages.**
  - Activities prevent causes of errors; detect and correct them early in the development process
  - QA substantially reduces the rate of products that do not qualify for shipment and/at the same time, reduce the costs of guaranteeing quality in most cases.

The objectives of SQA activities in Software Development

(1) Assuring an acceptable level of confidence that the software will conform to functional technical requirements.

(1) Assuring an acceptable level of confidence that the software will conform to managerial scheduling and budgetary requirements.

(3) Initiation and management of activities for the improvement and greater efficiency of software development and SQA activities.

The objectives of SQA activities in Software Maintenance (product-oriented)

(1) Assuring an acceptable level of confidence that the software maintenance activities will conform to the functional technical requirements.

(1) Assuring an acceptable level of confidence that the software maintenance activities will conform to managerial scheduling and budgetary requirements.

(3) Initiate and manage activities to improve and increase the efficiency of software maintenance and SQA activities.

Outline

1.1. What is software?

1.2. Classification of the causes of software errors

1.3. Software quality assurance – definition and objective

1.4. Software quality factors.

1.5. Factors affecting intensity of quality assurance

The Requirements Document

- Requirement Documentation (Specification) is one of the **most important elements** for achieving software quality
- Need to explore what constitutes a **good** software requirements document.
- Some SQA Models suggest 11-15 factors categorized; some fewer; some more
- Want to become **familiar** with these quality factors, and
- Who is really interested in them.
- **The need for comprehensive software quality requirements is pervasive in numerous case studies (see a few in this chapter).**
- (Where do the quality factors go??)

Need for Comprehensive Software Quality Requirements

- Need for improving poor requirements documents is widespread
- Frequently lack **quality factors** such as: usability, reusability, maintainability, …
- Software industry groups the long list of related attributes into what we call ***quality factors. (Sometimes non-functional requirements)***
- Natural to assume an unequal emphasis on all quality factors.
- Emphasis varies from project to project
  - Scalability; maintainability; reliability; portability; etc.
- Let's look at some of the categories…

Extra Thoughts

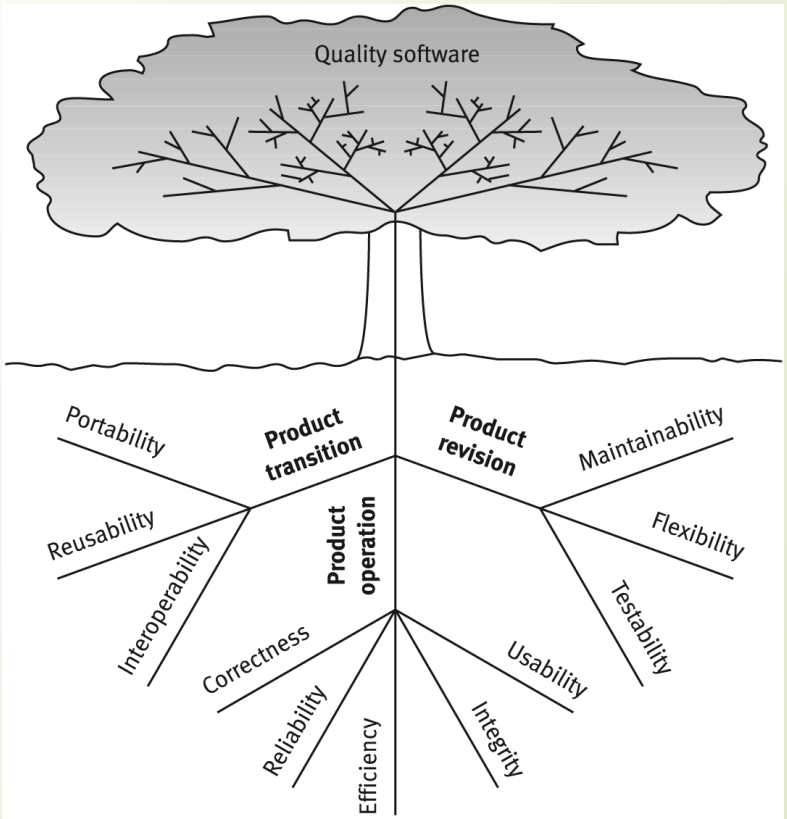
- Seems like in Software Engineering we concentrate on capturing, designing, implementing, and deploying with emphasis on **functional requirements.**
- Little (not none!) emphasis on the **non-functional requirements** (quality factors).
- More and more emphasis now placed on quality factors
- Can be a critical factor in satisfying overall requirements.

- In the RUP, non-functional requirements are captured in the Software Requirements Specification (SRS); functional requirement usually captured in Use Case stories.

McCall's Quality Factors

- McCall has 11 factors; Groups them into categories.
  - 1977; others have added, but this still prevail.
- Three categories:
  - **Product Operation Factors**
    - How well it runs….
    - Correctness, reliability, efficiency, integrity, and usability
  - **Product Revision Factors**
    - How well it can be changed, tested, and redeployed.
    - Maintainability; flexibility; testability
  - **Product Transition Factors**
    - How well it can be moved to different platforms and interface with other systems
    - Portability; Reusability; Interoperability
- Since these underpin the notion of quality factors and others who have added, reword or add one or two, we will spend time on these factors.

McCall's Quality Factors



Product operation factors

- Correctness
- Reliability
- Efficiency
- Integrity
- Usability

How well does it run and ease of use.

McCall's Quality Factors
Category: Product **Operation** Factors

1. Correctness.

- **Please note that we are asserting that 'correctness' issues are arising from the requirements documentation and the specification of the outputs…**
- **Examples include:**
  - **Specifying accuracies for correct outputs** at, say, NLT <1% errors, that could be affected by inaccurate data or faulty calculations;
  - **Specifying** the **completeness of the outputs** provided, which can be impacted by incomplete data (often done)
  - **Specifying the timeliness of the output** (time between event and its consideration by the software system)
  - **Specifying the standards** for coding and documenting the software system
  - we have talked about this: standards and integration; Essential!!

McCall's Quality Factors
Category: Product Operation Factors

2. Reliability Requirements. (remember, this quality factor is specified in the **specs**!)

- Reliability requirements deal with the failure to provide service.
  - Address failure rates either overall or to required functions.
- Example specs:
  - A heart monitoring system must have a failure rate of less than one per million cases.
  - **Downtime** for a system will not be more than ten minutes per month (me)
  - MTBF and MTTR - old and engineering, but still applicable.

3. Efficiency Requirements. Deals with the **hardware** resources needed to perform the functions of the software.

- Here we consider MIPS, MHz (cycles per second); data **storage** capabilities measured in MB or TB; communication lines (usually measured in KBPS, MBPS, or GBPS).
- Example spec: simply very slow communications…

McCall's Quality Factors
Category: Product Operation Factors

4. Integrity – deal with system security that prevent unauthorized persons access.

- Huge nowadays; Cyber Security; Internet security; network security, and more. These are certainly not the same!

5. Usability Requirements – deals with the scope of staff resources needed to train new employees and to operate the software system.

- Deals with learnability, utility, usability, and more. (me)
- Example spec: A staff member should be able to process n transactions / unit time. (me)

Product revision factors

- Maintainability
- Flexibility
- Testability

Can I fix it easily, retest, version it, and deploy it easily?

McCall's Quality Factors

Category: Product **Revision** Software Factors

These deal with requirements that affect the complete range of software maintenance activities:

- corrective maintenance,
- adaptive maintenance, and
- perfective maintenance
- KNOW THE DIFFERENCES!

- **1. Maintainability Requirements**
  - The degree of effort needed to identify reasons (find the problem) for software failure and to correct failures and to verify the success of the corrections.
  - Deals with the modular structure of the software, internal program documentation, programmer manual, architectural and detail design and corresponding documentation
  - Example specs: size of module <= 30 statements.
  - Refactoring...

McCall's Quality Factors
Category: Product Revision Software Factors

2. Flexibility Requirements – deals with resources to change (adopt) software to different types of customers that use the app perhaps a little differently;

- May also involve a little perfective maintenance to perhaps do a little better due to the customer's perhaps slightly more robust environment.
- Different clients exercise software differently. This is big!

3. Testability Requirements –

- Are intermediate results of computations predefined to assist testing?
- Are log files created? Backup?
- Does the software diagnose itself prior to and perhaps during operations?

Product transition factors

- Portability
- Reusability
- Interoperability

Can I move the app to different hardware? Interface easily with different hardware / software systems; can I reuse major portions of the code with little modification to develop new apps?

McCall's Quality Factors
Category: Product **Transition** Software Quality Factors

1. Portability Requirements: If the software must be ported to different environments (different hardware, operating systems, …) and still maintain an existing environment, then portability is a must.

2. Reusability Requirements: Are we able to reuse **parts** of the app for new applications?

- Can save immense development costs due to errors found / tested.
- Certainly higher quality software and development more quickly results.
- Very big deal nowadays.

McCall's Quality Factors
Category: Product Transition Software Quality Factors

3. Interoperability Requirements: Does the application need to **interface** with other existing systems

- Frequently these will be known ahead of time and plans can be made to provide for this requirement during design time.

  Sometimes these systems can be quite different; different platforms, different databases, and more

- Also, industry or standard application structures in areas can be specified as requirements.

Alternatives

- Some other SQA professionals have offered essentially renamed quality factors.
- One has offered 12 factors; another 15 factors.

- Totally five new factors were suggested
- Evans and Marciniak offer two 'new' ones:
  - Verifiability and Expandability
- Deutsch and Willis offer three 'new' ones.
  - Safety
  - Manageability, and
  - Survivability

McCall's factor model and alternative models

No.	Software quality factor	McCall's classic model	Alternative factor models	
			Evans and Marciniak model	Deutsch and Willis model
1	Correctness	+	+	+
2	Reliability	+	+	+
3	Efficiency	+	+	+
4	Integrity	+	+	+
5	Usability	+	+	+
6	Maintainability	+	+	+
7	Flexibility	+	+	+
8	Testability	+		
9	Portability	+	+	+
10	Reusability	+	+	+
11	Interoperability	+	+	+
12	Verifiability		+	+
13	Expandability		+	+
14	Safety			+
15	Manageability			+
16	Survivability			+

Outline

1.1. What is software?

1.2. Classification of the causes of software errors

1.3. Software quality assurance – definition and objective

1.4. Software quality factors.

1.5. Factors affecting intensity of quality assurance

Factors affecting Intensity of SQA Activities

- SQA Activities are linked to the completion of a project phase
  - Requirements, design, etc.
- The SQA activities need to be <u>integrated</u> into the development plan that implements one or more software development models, such as the waterfall, prototyping, spiral, …
- They need to be activities just like other more traditional activities – be entered in plan, scheduled, etc.

Factors affecting Intensity of SQA Activities

- SQA planners need to determine
  - A list of SQA activities needed for the project
  - And then for each activity, they need to decide on
    - Timing
    - Type of QA activity to be applied (there are several)
    - Who performs the activity and resources required.
      - Important to note that many participate in SQA activities
        - » Development team
        - » Department staff members
        - » Independent bodies
    - Resources required for the removal of defects and introduction of changes.

Factors affecting Intensity of SQA Activities

- Sad testimony that few want to allocate the necessary time for SQA activities.
  - This means time for SQA activities and then time for subsequent removal of defects.
  - Often, there is no time for follow-on work!!
- Activities are not simply cranked in and absorbed!
- So, time for SQA activities and defect correction actions needs to be examined.

Factors Affecting the Required Intensity of QA Activity

- Project Factors
  - Magnitude of the project – how big is it?
  - Technical complexity and difficulty
  - Extent of reusable software components – a real factor
  - Severity of failure outcomes if project fails – essential!
- Team Factors
  - Professional qualifications of the team members
  - Team acquaintance w/ project and experience in the area
  - Availability of staff members who can professionally support the team, and
  - Percentage of new staff members in the team.